

Architectural Analysis of Freeplane – Reverse Engineering and Evaluation

1. Introduction and Analysis Methodology This report presents an architectural analysis of Freeplane with the objective of identifying its main structural characteristics, architectural style, and design principles. The analysis aims to reconstruct the system architecture and understand how its components are organized and interact. The reverse engineering process was conducted using static code analysis, documentation review and statistics gathered directly from the official Freeplane repository. The codebase was examined to identify packages, modules, and dependencies, while repository documentation was used to support architectural interpretation.

The system can be described as an **imperfect micro-kernel (plug-in) monolith**; where extensible components are connected to the core system through the OSGi plugin framework an industry-standard solution for blending separate components in a single entry point from the user standpoint. However, the core module is not just a shell, and it provides essential services, user interface management, and application coordination, while plugins extend the system with additional functionalities. Plugins represent extensions: they do not make the software, but they extend it with advanced features. This pattern breaks some of the micro-kernel architectural style principles, thus the *imperfect* definition.

This report will delve into architectural details, showing developers' choices, their compliance to modern architectural styles and design patterns: the SOLID principles and the Clean Architecture theory will be benchmarks to assess overall Freeplane's architecture health. The C4 notation will help in detailing better software functionalities and how they interact in its environment. C4 diagrams have been written in PlantUML and graphically formatted with the help of an LLM.

2. The System in its Ecosystem: C4 Context Model Freeplane was born as a fork of the well-known Freemind software. The official documentation reports that the decision was taken to improve software's design and to speed up its development and maintenance cycles.

The context diagram illustrates how Freeplane operates within its external environment and interacts with both users and supporting systems. Two main user roles are identified. Beginner users interact with the system to create and manage mind maps using basic functionality, while advanced users extend the system through scripting and plugins, enabling automation and customization.

The software presents itself as a mind-mapping tool, and it features a proprietary file format `.mm`. The software stores such files in the computer File System, in a user-defined directory. The file system, and more generally the OS, represent the software's persistence layer. UI personalization, such as themes, font, are saved in a dedicated repository within the computer filesystem. This interaction

reflects the system’s reliance on local persistence rather than external databases.

To enhance its mind-mapping toolset, web browsers are invoked to open hyperlinks embedded within maps. Users can add them to nodes. When clicked, a browser instance is opened to show the webpage the URL points to. This can be mostly useful to include sources or images directly from the internet in the mind-map.

Freeplane grew to become a more and more comprehensive tool, to help students, professors, and professionals from various fields requiring productivity tools: to do so, Freeplane supports built-in Email support; with a single click users can be directly redirected to their main Email provider. It also provides native support for TaskJuggler, a project-management tool: mindmaps can be transformed in tasks within the external software; this features enhance productivity for busy professionals such as Project Managers. Whereas the interaction with the email system is just a redirect, the software actively transfer data to TaskJuggler, making the transition to the new software smooth and fast. Freeplane supports built-in LLM interaction. Through a dedicated plugin, users can link the software to their favorite LLM system. The software allows AI tools to directly interact with mindmaps.

All these features show how the software is more than just a mind-mapping tool, but it can be described as a productivity software: mindmaps can be useful for a large variety of users, and productivity features make the software more appealing to highly professional users looking for ways to speed up their workflow.

3. Decomposition and Runtime: C4 Container Model (Target: ~400 words)

- **Objective:** Show the deployable/executable units. For a desktop app like Freeplane, the “containers” are typically the base framework, the Core Engine, and the bundles/plugins.
- **Action:** *[Insert C4 Level 2 Diagram: Container]*
- **Questions to answer:**
 - How is the runtime structured? What is the role of the plugin framework (e.g., OSGi / Equinox) that acts as the host?
 - Which logical/physical modules make up the unalterable “Core,” and which are identified as external “Plugins” (e.g., PDF export plugin, scripting plugin, LaTeX plugin)?
 - How do these containers communicate at runtime (e.g., separate classloading, direct in-memory calls)?

4. Mapping to Clean Architecture: Theory vs. Reality (Target: ~500 words)

- **Objective:** Apply the lens of Clean Architecture to Freeplane’s source code. Since this is legacy software, it is crucial to highlight both adherence and violations.

- **Questions to answer:**
 - Which classes/packages in Freeplane represent the *Entities* (e.g., the logical model of the node, the map) and the *Use Cases* (e.g., tree manipulation actions)?
 - Are the user interface layer (Swing/AWT) and the persistence layer (XML parser) truly relegated to the outer layers (*Interface Adapters* / *Frameworks*) as Clean Architecture dictates?
 - **Critical Evaluation:** Is the *Dependency Rule* strictly followed? Or did you find instances where the inner domain (Core/Entities) has direct dependencies on GUI elements or external libraries?

5. Zooming into the Engine: C4 Component Model of the Core (Target: ~600 words)

- **Objective:** Detail the internal design of the Core and showcase the extension points.
- **Action:** *[Insert C4 Level 3 Diagram: Component for the main Controller and Plugin Management]*
- **Questions to answer:**
 - How is the typical MVC (Model-View-Controller) pattern of Freeplane (e.g., `MapModel`, `NodeModel`, `ModeController`) structured at the component level?
 - What is the anatomy of the internal Plugin Manager? How does it discover, load, and register a plugin within the application's lifecycle?
 - Which architectural components are responsible for event dispatching (e.g., model changes that need to update the UI and notify plugins)?

6. Boundary Management and Core-Plugin Interaction (Target: ~400 words)

- **Objective:** Explain the engineering mechanisms by which plugins cross architectural boundaries to extend Freeplane without modifying its core.
- **Questions to answer:**
 - How are the *Boundaries* between the Core and a generic Plugin defined? (e.g., via pure Java interfaces abstracted in the Core).
 - How does Freeplane apply the *Dependency Inversion Principle (DIP)* to ensure that plugins depend on the Core, but not vice versa?
 - Do plugins interact with the Core by sending DTOs (Data Transfer Objects), or do they have direct write access to domain objects (Entities)? (*Note: This is a common architectural vulnerability in desktop software*).
 - How does the *Event Delegation* / *Listeners* mechanism work to bridge boundaries when a modification occurs on the map?

7. Architectural Evaluation and Conclusions (Target: ~100 words)

- **Objective:** Summarize your analysis with a critical eye.

- **Questions to answer:**
 - In light of your analysis, how robust, maintainable, and extensible is Freeplane's architecture?
 - If you had to propose an architectural refactoring based on Clean Architecture principles, which part would you isolate first (e.g., stronger decoupling between the Swing UI and the map logic)?